

Trabajo Práctico N.º 3

Introducción al Shell de [MongoDB](#) (mongosh)

Materia	Bases de Datos Avanzadas
Tema	Conexión, navegación, altas y consultas básicas desde la línea de comandos
Modalidad de trabajo	Individual o en parejas
Herramientas	MongoDB Community / MongoDB Atlas, mongosh, MongoDB Compass

1. Introducción

Mongosh (MongoDB Shell) es el cliente interactivo de línea de comandos oficial de MongoDB. Permite conectarse a una instancia (local o en la nube, como MongoDB Atlas), ejecutar comandos administrativos y operar sobre los datos utilizando una API basada en JavaScript. Dominar el shell es fundamental porque es la herramienta más directa para entender qué ocurre "por debajo" de cualquier interfaz gráfica, y es también la que se emplea para automatizar tareas mediante scripts.

Este trabajo práctico retoma los documentos diseñados en el TP N.º 1 y los lleva, por primera vez, a una instancia real de MongoDB a través del shell.

2. Objetivos de aprendizaje

- Instalar y/o conectar mongosh a una instancia de MongoDB local o a un clúster de MongoDB Atlas.
- Utilizar los comandos de navegación del shell para listar bases de datos, seleccionar una base y listar colecciones.
- Comprender que en MongoDB una base de datos y una colección se crean de forma implícita en la primera escritura.
- Insertar documentos individuales y en lote mediante `insertOne()` e `insertMany()`.
- Realizar consultas básicas con `find()` y `findOne()`.
- Escribir y ejecutar un script .js de carga inicial de datos (seed script).

3. Marco teórico

3.1 ¿Qué es mongosh?

Mongosh es la evolución del shell clásico de MongoDB (mongo, ya deprecado). Es un entorno REPL (Read-Eval-Print Loop) que interpreta JavaScript y expone un objeto global `db` para interactuar con la base de datos activa. También soporta autocompletado, resaltado de sintaxis y la ejecución de archivos de script completos.

3.2 Instalación y conexión

Existen dos escenarios habituales de trabajo para este práctico; utilice el que la cátedra indique o el que tenga disponible:

- **Instancia local:** instalar MongoDB Community Server y mongosh, y conectarse con el comando `mongosh` sin argumentos (asume `localhost:27017`).
- **MongoDB Atlas (nube, gratuito):** crear un clúster gratuito (M0), obtener la cadena de conexión desde el botón "Connect" y conectarse con mongosh "`<connection string>`".

```
# Conexión local
$ mongosh

# Conexión a Atlas
$ mongosh "mongodb+srv://cluster0.xxxxx.mongodb.net/" --apiVersion 1 --username <usuario>
```

3.3 Comandos de navegación

Una vez dentro del shell, se dispone de comandos "de ayudante" (no son JavaScript puro, sino atajos del propio shell) para explorar el servidor:

Comando	Descripción	Ejemplo
<code>show dbs</code>	Lista las bases de datos existentes en el servidor.	<code>show dbs</code>
<code>use <db></code>	Cambia (o crea, si no existe) la base de datos activa.	<code>use tienda</code>
<code>db</code>	Muestra el nombre de la base de datos activa.	<code>db</code>
<code>show collections</code>	Lista las colecciones de la base activa.	<code>show collections</code>
<code>db.stats()</code>	Muestra estadísticas de la base activa (tamaño, cantidad de colecciones, etc.).	<code>db.stats()</code>
<code>db.<col>.countDocuments()</code>	Cuenta los documentos de una colección.	<code>db.estudiantes.countDocuments()</code>
<code>help</code>	Muestra la ayuda general del shell.	<code>help</code>
<code>exit</code>	Cierra la sesión del shell.	<code>exit</code>

Importante

MongoDB no crea físicamente una base de datos ni una colección hasta que se inserta al menos un documento en ellas. Por eso, tras un `use tienda`, la base "tienda" no aparecerá en `show dbs` hasta que se ejecute la primera inserción.

3.4 Sintaxis general de operación sobre colecciones

Toda operación sobre datos sigue el patrón `db.<nombreColeccion>.<metodo>(<argumentos>)`. Por ejemplo, `db.estudiantes.find({})` consulta todos los documentos de la colección `estudiantes`.

3.5 Inserción de documentos

Comando	Descripción	Ejemplo
insertOne(doc)	Inserta un único documento; retorna el <code>_id</code> generado.	<code>db.estudiantes.insertOne({...})</code>
insertMany([docs])	Inserta un array de documentos en una sola operación.	<code>db.estudiantes.insertMany([({...}])</code>

Ejemplo de inserción de un único documento (retomando el modelo de "estudiante" del TP N.º 1):

```
use badavanzada

db.estudiantes.insertOne({
  legajo: "L-2044",
  nombre: "Sofía Herrera",
  edad: 22,
  activo: true,
  materiasCursadas: ["Bases de Datos I", "Programación II"],
  tutor: null
})
```

Si el `_id` no se especifica, MongoDB genera automáticamente un `ObjectId` único para el documento.

3.6 Consultas básicas

Comando	Descripción	Ejemplo
find()	Devuelve un cursor con todos los documentos que cumplen el filtro (vacío = todos).	<code>db.estudiantes.find()</code>
find(filtro)	Filtra documentos por igualdad de campos.	<code>db.estudiantes.find({activo:true})</code>
findOne(filtro)	Devuelve el primer documento que cumple el filtro.	<code>db.estudiantes.findOne({legajo:"L-2044"})</code>
.pretty()	Formatea la salida del cursor con indentación legible.	<code>db.estudiantes.find().pretty()</code>

3.7 Scripts de carga (seed scripts)

Para no reescribir manualmente decenas de inserciones cada vez que se necesita una base de datos de prueba, es una práctica habitual escribir un archivo `.js` con toda la secuencia de comandos y ejecutarlo de una sola vez.

```
# Desde la terminal del sistema operativo (no desde dentro del shell)
$ mongosh badavanzada seed_estudiantes.js

# O bien, ya conectado dentro del shell:
test> load("seed_estudiantes.js")
```

4. Consignas de trabajo

Documente cada paso de esta guía con capturas de pantalla de la terminal (o registrando la sesión con script/typescript) y entregue, además, todos los comandos ejecutados en un único archivo .js reproducible.

Ejercicio 1 — Instalación y primera conexión

Instale MongoDB Community Server y mongosh (o cree un clúster gratuito en MongoDB Atlas). Conéctese exitosamente al servidor y capture el prompt inicial del shell.

Ejercicio 2 — Creación implícita de una base de datos

Ejecute use badavanzada, confirme con el comando db que la base activa cambió, y luego ejecute show dbs. Explique por escrito por qué la base "badavanzada" no aparece todavía en el listado.

Ejercicio 3 — Primera alta con insertOne

Utilizando el documento JSON de "estudiante" que diseñó en el Ejercicio 2 del TP N.º 1, insértelo en la colección estudiantes con insertOne(). Vuelva a ejecutar show dbs y show collections y verifique que ahora sí aparecen la base y la colección.

Ejercicio 4 — Alta masiva con insertMany

Amplíe el modelo anterior a al menos 6 estudiantes distintos (variando edad, materias cursadas y estado activo) e insértelos todos en una sola operación con insertMany(). Ejecute countDocuments() para confirmar la cantidad total de documentos en la colección.

Ejercicio 5 — Consultas básicas

Escriba y ejecute: (a) una consulta que devuelva todos los documentos de la colección; (b) una consulta que filtre solamente los estudiantes con activo: true; (c) una consulta con findOne() que busque un estudiante por su legajo. Capture la salida de cada una.

Ejercicio 6 — Exploración del servidor

Ejecute db.stats() sobre la base badavanzada y explique en un párrafo qué información relevante para un administrador de bases de datos provee esta salida (tamaño de datos, cantidad de colecciones, índices, etc.).

Ejercicio 7 — Script de carga reproducible

Cree un archivo seed_badavanzada.js que, ejecutado con mongosh badavanzada seed_badavanzada.js desde una base vacía, deje creada la colección estudiantes con los mismos documentos del Ejercicio 4.

El script debe comenzar eliminando la colección si ya existe (db.estudiantes.drop()) para garantizar que la ejecución sea idempotente (es decir, que pueda correrse varias veces sin duplicar datos).

```
// seed_badavanzada.js
db.estudiantes.drop();

db.estudiantes.insertMany([
  { legajo: "L-2044", nombre: "Sofía Herrera", edad: 22, activo: true, materiasCursadas:
["Bases de Datos I"] },
  // ... resto de los documentos
]);

print("Carga finalizada. Documentos insertados: " + db.estudiantes.countDocuments());
```

5. Forma de entrega

- Archivo seed_badavanzada.js con el script de carga completo del Ejercicio 7.
- Documento (PDF o Word) con las capturas de pantalla de cada ejercicio y las explicaciones solicitadas.
- Plazo de entrega y vía de entrega: a definir por la cátedra en el aula virtual.

6. Criterios de evaluación

Criterio	Descripción	Puntos
Instalación y conexión	El estudiante logra conectarse correctamente al shell, local o en Atlas.	15
Comandos de navegación	Uso correcto de use, show dbs, show collections, db.stats().	15
Altas por shell	insertOne() e insertMany() ejecutados correctamente con datos coherentes con el TP N.º 1.	25
Consultas básicas	find(), findOne() y filtros simples ejecutados y correctamente interpretados.	20
Script de carga	El script es idempotente, reproducible y bien comentado.	25
Total		100